By TOUCH DEN · November 11, 2026

Representing Text

Video 1 · How Text Becomes Numbers

How Computers Work · Section 3

In this video, you will learn:

- How text becomes numbers (character encoding)
- ASCII, and why `H` is `0x48`
- Unicode and UTF-8, for every language

How text becomes numbers

A computer stores only numbers, so each character is stored as a number everyone agrees on, a **character encoding** (a shared codebook).

- Writer turns `H` into a number using the table; reader turns it back with the **same** table.
- Different tables → **garbled text**.

So "why is `H` the byte `0x48`?" → **because a standard table (ASCII) says so**, and everyone agreed.

ASCII

ASCII (1960s): a number 0–127 for each basic English character. Fits in 7 bits.

```
$ python3 -c 'print(ord("H"), hex(ord("H")))
print(ord("A"), ord("a"), ord("0"), ord(" "))'
72 0x48
65 97 48 32
```

- `H` = 72 = `0x48` (the lesson-2 dump!).
- **The character `'0'` is 48, not 0.** Different things!
- Upper vs lower differ by 32: `A` =65, `a` =97.

128 characters can't hold the world, no `é`, no Khmer.

Unicode and UTF-8

Unicode: one giant table; every character gets a unique number (**code point**). ASCII kept at the bottom.

UTF-8: the rule to store code points as bytes.

- **ASCII-compatible:** 0–127 stay one byte.
- **More bytes only when needed:** 2–4 bytes for the rest.

```
$ python3 -c 'print("A".encode("utf-8").hex(" "))
print("é".encode("utf-8").hex(" "))
print("ก".encode("utf-8").hex(" "))'
41
c3 a9
e1 9e 80
```

One character is not always one byte. `ก` is three bytes.

Recap: text becomes numbers

- A **character encoding** is a shared table mapping characters to bytes.
- **ASCII:** basic English, 0–127; `H` = `0x48`; `'0'` = 48, not 0.
- **Unicode + UTF-8:** every language; ASCII-compatible; **one char ≠ one byte** (Khmer = 3 bytes).

Next video: Base64, URL encoding, and why encoding ≠ protection.

Representing Text

Video 2 · Base64, URL Encoding, and Why It Matters

How Computers Work · Section 3

In this video, you will learn:

- Base64, and why it is NOT encryption
- URL (percent) encoding
- Why encoding matters in security

Base64 is encoding, NOT encryption

Base64 packs any binary data into 64 safe printable characters, to pass through text-only channels.

```
$ printf 'Mina Coffee' | base64
TWluYSBDb2ZmZWU=
$ printf 'TWluYSBDb2ZmZWU=' | base64 -d
Mina Coffee
```

Recognize it: letters/digits, often ending in `=`.

> **Base64 gives no secrecy. Anyone decodes it instantly, no key.**

Seeing Base64 in a token does **not** mean the data is protected.

Encoding vs encryption

The most important distinction in this lesson:

- **Encoding** (ASCII, UTF-8, Base64, URL): a **public rule** to represent data. No secret, reversible by anyone.
- **Encryption** (Cryptography course): a **secret key** makes data unreadable without it.

Whenever you see encoded data, read it; never assume it's protected. Never "secure" a secret by encoding it.

URL (percent) encoding

URLs allow only limited characters. Others become `%` + the byte in hex.

```
$ python3 -c 'import urllib.parse
print(urllib.parse.quote("a b&c=/?"))'
a%20b%26c%3D/%3F
```

- space → `%20` (space = byte `0x20` = 32, the ASCII space!)
- `&` → `%26`, `=` → `%3D`, `?` → `%3F`

Attackers use it to **sneak characters past filters**: a filter missing `<` may miss `%3C`, which the browser turns back into `<`.

Recap: Base64, URL, and security

- **Base64** = encoding, **not** encryption; decodable by anyone, instantly.
- **Encoding is a public rule; encryption needs a secret key.** Encoding never protects.
- **URL encoding** = `%` + hex byte; used to bypass filters.
- **Ambiguity is an attack surface:** same bytes read two ways = injection and filter bypass.

Next lesson: the machine that works on all these bytes, the CPU, memory, and storage.